

深圳大学实验报告

课程名称： 计算机网络

实验项目名称： 实验3 数据包抓取与分析

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 王璐

报告人： 汉雨 学号： 2024040042 班
级：

实验时间： 20260412

实验报告提交时间：

实验目的

学习安装、使用协议分析软件，掌握基本的数据报抓取、过滤和分析方法，能分析 HTTP、TCP、ICMP 等协议。

实验环境

1. 使用具有 Internet 连接的 Windows 操作系统；
2. 抓包软件 Wireshark。

实验内容：

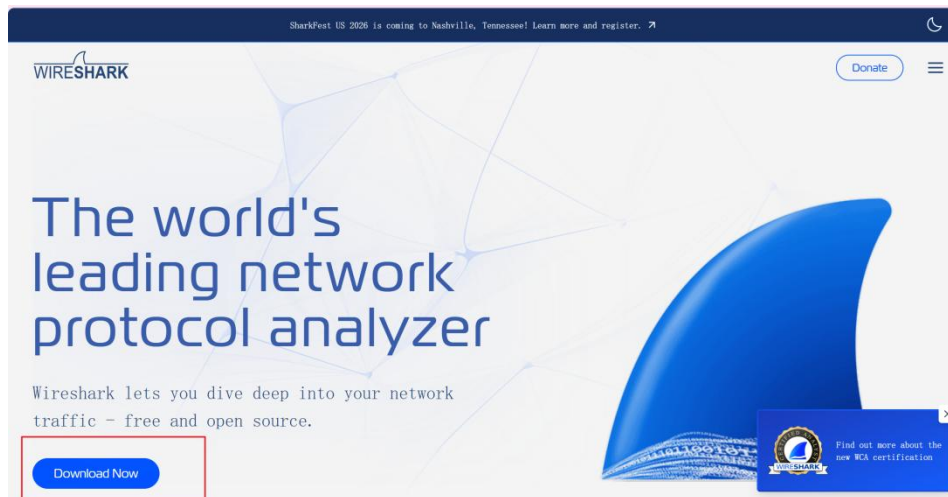
3. 安装学习 Wireshark 软件；
4. 抓包与分析 HTTP 协议；
5. 分析 TCP 协议；
6. 分析 TCP 三次握手；
7. 分析 ICMP 协议。

实验步骤：

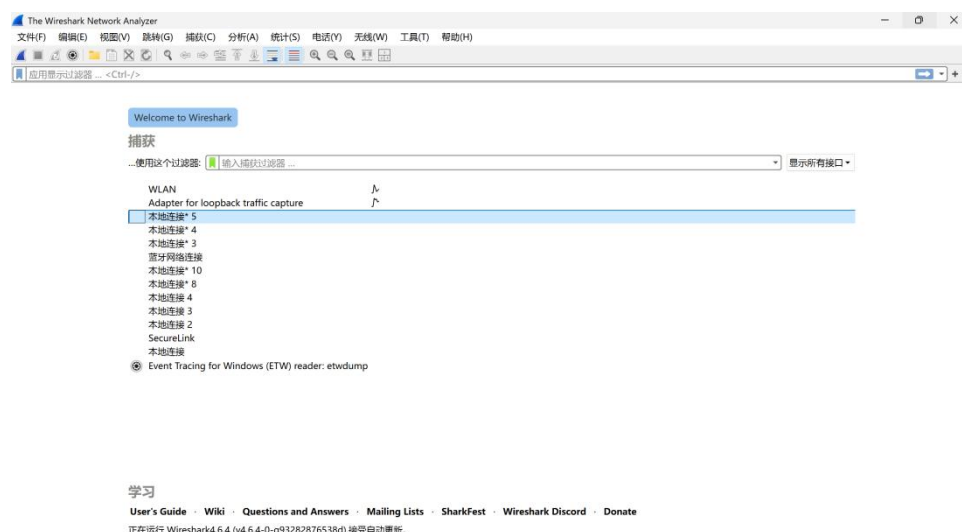
（用文字描述实验过程，并用截图辅助说明）

一、 安装学习 Wireshark 软件

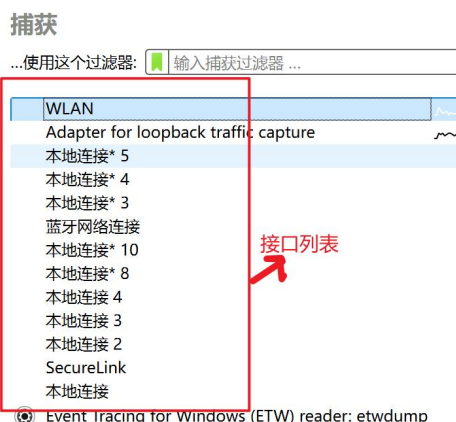
- 按照教学指导上给的 Wireshark 官方网址 <https://www.wireshark.org/> 下载软件



获取安装包后，一路点击“Next”即可。
此为下载完成后的主界面

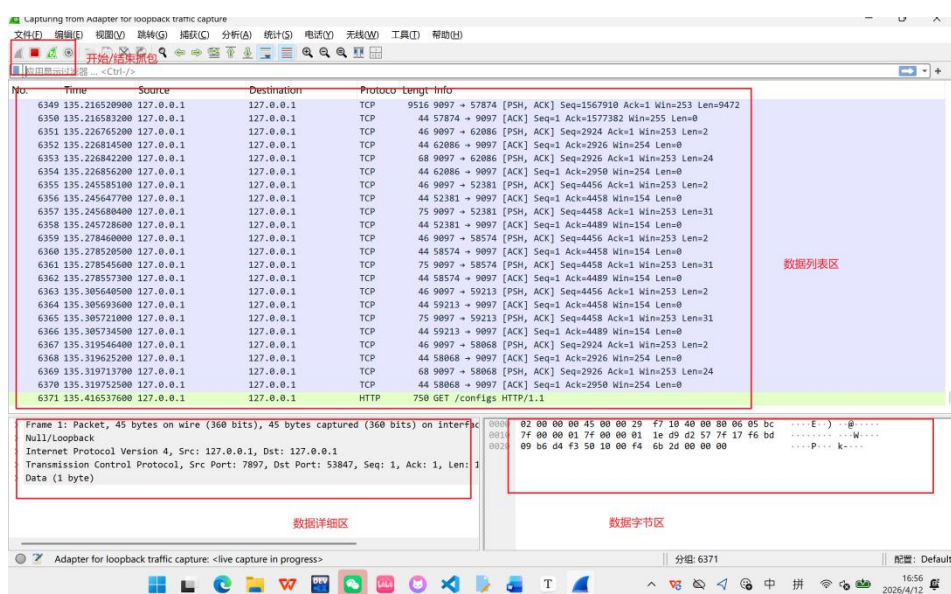


- 学习 wireshark 软件
点开主界面可得到接口列表：



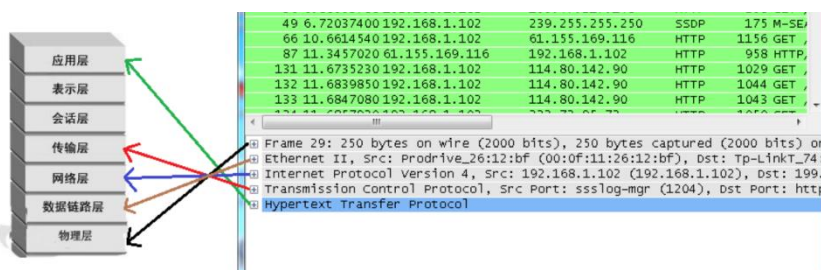
选择想要捕获的接口，点击右键，出现 Start Capture(开始捕获)，点击 Start Capture(开始捕获)，即可进行捕获该网络信息，开始抓取网络包。

随后会出现这样的界面。



界面包含 Display Filter(显示过滤器)，过滤器主要分为显示过滤器和捕获过滤器两类。显示过滤器（Display Filter）用于对已经捕获的数据进行筛选，是 Wireshark 中最常用的过滤方式，通常在界面顶部输入框中使用，其语法较为灵活，例如输入 `http` 可以仅显示 HTTP 协议的数据包；捕获过滤器（Capture Filter）则是在抓包之前设置条件，仅捕获符合要求的数据包，其语法相对简单，例如 `port 80` 表示只捕获端口为 80 的数据。在显示过滤器中，可以通过多种方式进行筛选，如按协议过滤（`http`、`tcp`、`udp`、`dns`、`icmp`），按 IP 地址过滤（`ip.addr == 192.168.1.1` 表示筛选所有与该 IP 相关的数据包，`ip.src` 和 `ip.dst` 分别表示源地址和目的地址），按端口过滤（`tcp.port == 80`、`tcp.srcport == 443`、`tcp.dstport == 80`），按域名过滤（`http.host == "www.baidu.com"`），以及按关键字过滤（`frame contains "login"` 用于查找包含特定字符串的数据包）。

Packet Details Pane(数据包详细信息)：用于按协议分层展示数据包内容，对应计算机网络的五层结构。最底层为物理层，在 Wireshark 中通常以 Frame 形式体现，包含数据包长度和时间等信息；其上是数据链路层，对应 Ethernet 协议，包含源 MAC 和目的 MAC 地址；再上是网络层，对应 IP 协议，包含源 IP 和目的 IP 地址；然后是传输层，对应 TCP 或 UDP 协议，包含端口号及相关控制信息；最上层是应用层，对应 HTTP、DNS 等协议，包含具体的应用数据内容。通过分组详情栏可以直观地观察数据包的分层结构及封装过程。

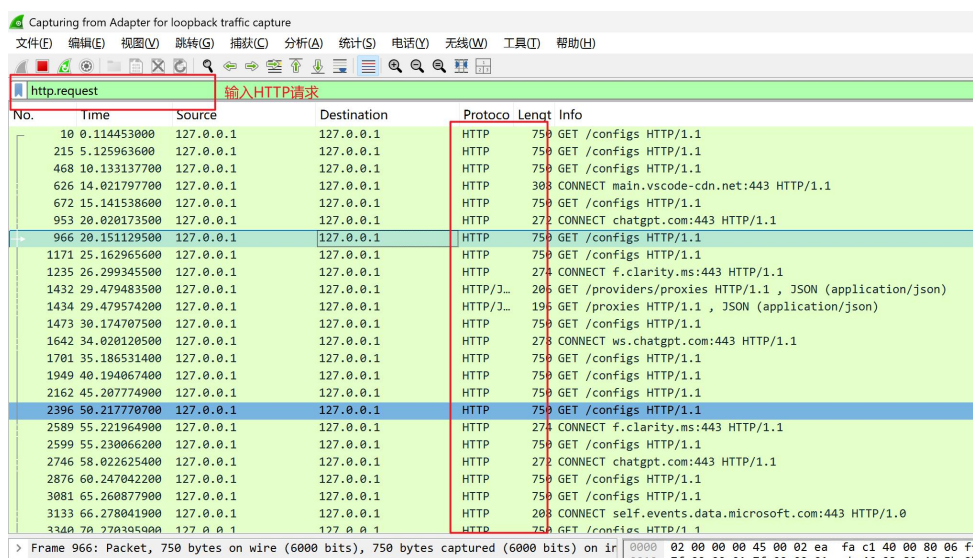


二、抓包与分析 HTTP 协议

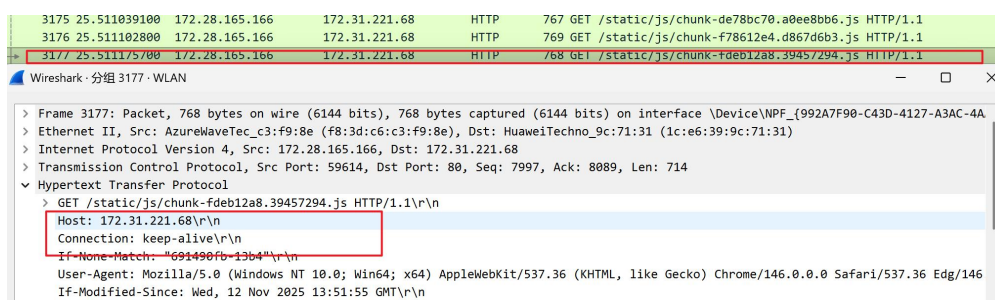
➤ 过滤 http 协议的分组

可以先在浏览器中输入一个网址。(http://172.31.221.68)

在过滤器中输入“http”



单击分组，可以得知分组的详细信息。获知此次通信的源 IP 地址和目的 IP 地址（172.28.165.166/172.31.221.68）

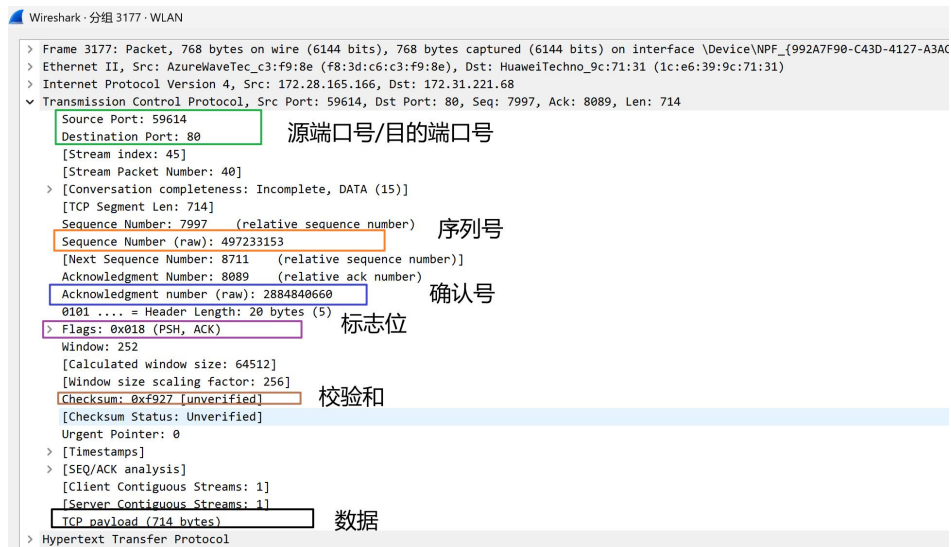


三、分析 TCP 协议

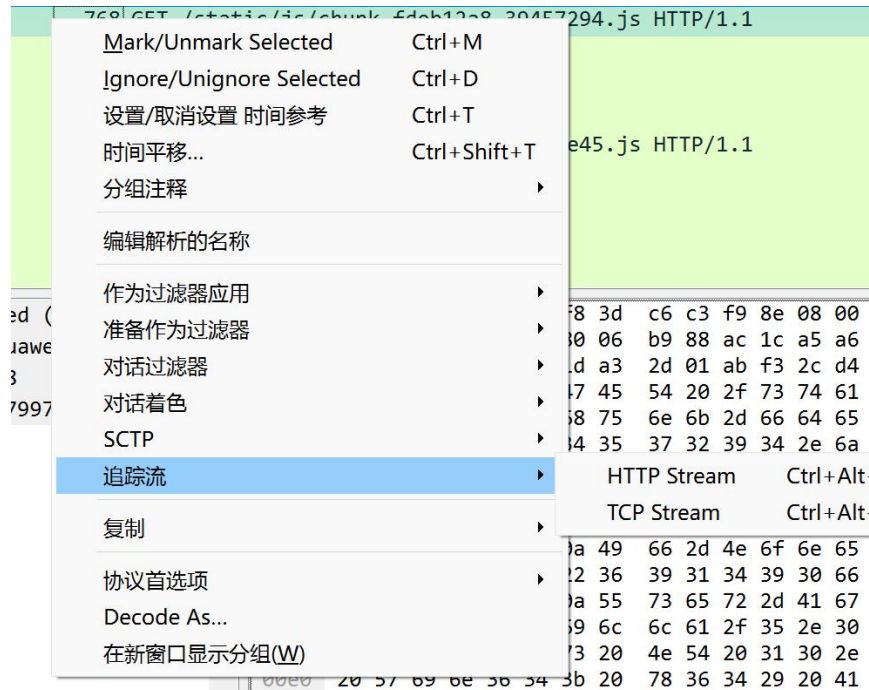
报文的源端口为 59614，是客户端发起通信的临时随机端口，目的端口为 80，指向 HTTP 标准服务端口，表明这是一次 Web 服务通信。传输层头部长度为 20 字节，为标准 TCP 头部长度。序列号 497233153 标识了本报文数据段的第一个字节编号，确认号 8089 则表示接收方已收到编号小于该值的所有数据，期望接收后续从 8089 开始的数据，实现可靠传输的确认机制。

报文标志位为 0x018，对应 PSH 和 ACK 两个标志位均置 1：ACK 确认位为 1，说明这是一个接收方发出的确认报文；PSH 推送位为 1，指示接收方收到数据后需立即提交给应用层，而非缓存等待。窗口大小为 256 字节，体现了接收方的流量控制能力，告知发送方自身当前还能接收 256 字节数据。校验和 0xf927 用于验证报文传输完整性，若接收方计算值不符则丢弃报文。报文还承载了 714 字节的 TCP payload，经解析为超文本传输协议数据，即服务器向客户端返回的 HTTP 响应内容

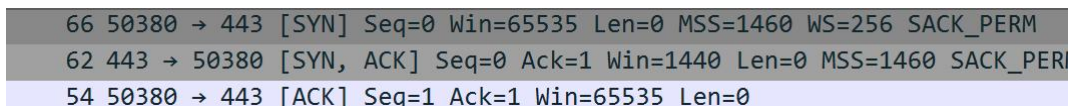
- 可获得 TCP 协议信息



- 单击右键选择 TCP 流



- 出 TCP 建立联系, 出现 SYN 标志

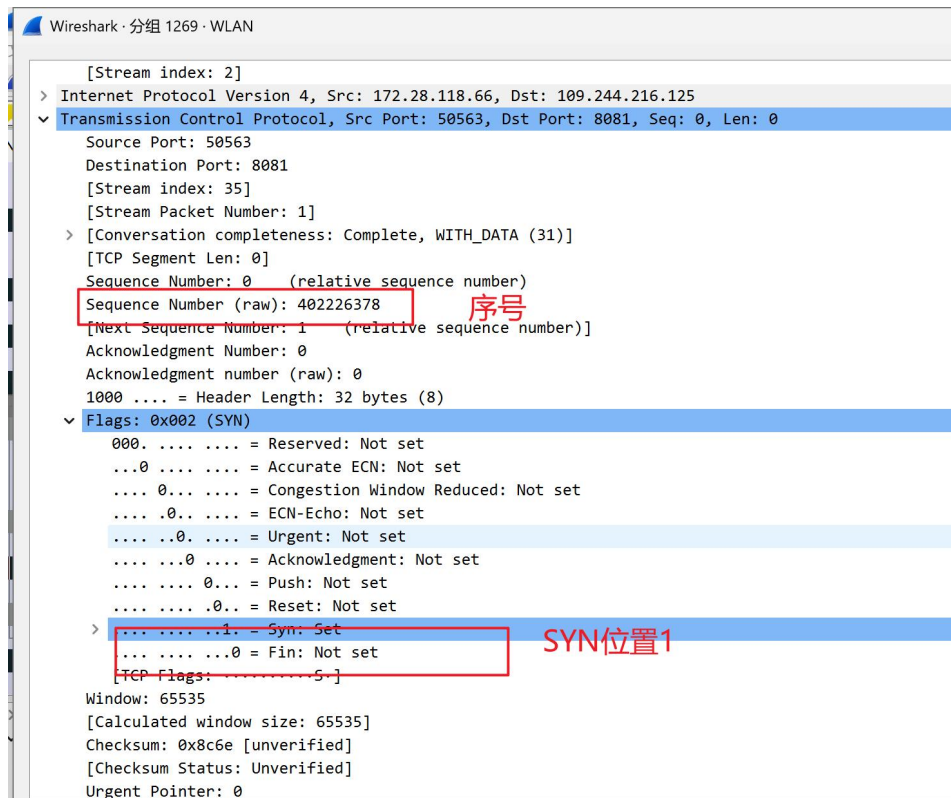


四、分析 TCP 三次握手

TCP 三次握手是建立可靠连接的核心流程，本质是客户端和服务端互相确认「对方的发送能力、接收能力正常」，并同步初始序列号（ISN），为后续可靠传输打基础。下面结合 SYN、SYN-ACK、ACK 三个核心报文，拆解每一步的序号（Sequence Number, SEQ）和确认号（Acknowledgment Number, ACK）逻辑。

- 第一次握手 SYN

- ✓ 第一次握手：客户端 → 服务器（SYN 报文）
- ✓ 标志位：SYN=1, ACK=0
- ✓ 序号（SEQ）：x（客户端的初始序列号 ISN_C）
- ✓ 确认号（ACK）：无意义（ACK=0，接收方忽略）
- ✓ 作用：客户端向服务器发起连接请求，同步自己的初始序列号，告知服务器「我要建立连接，我的初始序号是 x」。
- ✓ 细节：SYN 报文不携带应用数据，但占用 1 个序号，因此客户端下一个报文的序号会是 x+1。



- 第二次握手(SYNACK)。

- 标志位：SYN=1, ACK=1
- 序号（SEQ）：y（服务器的初始序列号 ISN_S）
- 确认号（ACK）：x+1（确认收到客户端的 SYN：x+1，表示「我已收到你序号 x 的 SYN，下一个期望收到 x+1 开始的数据」）
- 作用：服务器同时完成两件事：
用 SYN=1 同步自己的初始序列号，同意建立连接；
用 ACK=1 和确认号 x+1，确认客户端的连接请求有效。
- 细节：服务器的 SYN 同样占用 1 个序号，因此服务器下一个报文的序号会是 y+1。

```

Transmission Control Protocol, Src Port: 8081, Dst Port: 50563, Seq: 0, Ack: 1, L
Source Port: 8081
Destination Port: 50563
[Stream index: 35]
[Stream Packet Number: 2]
> [Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 71563853 序号
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 402226379 确认号是第一次握手
1000 .... = Header Length: 32 bytes (8) 序号加1
Flags: 0x012 (SYN, ACK)
000. .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
... 0... = Congestion Window Reduced: Not set
.... 0... = ECN-Echo: Not set
.....0 = Urgent: Not set
.... 1... = Acknowledgment: Set ACK位置1
.... 0... = Push: Not set
.... 0... = Reset: Not set
> .... 1... = Syn: Set SYN位置1
.... 0... = Fin: Not set
[TCP Flags: .....A..S.]

```

● 第三次握手 (ACK)

- 标志位: SYN=0, ACK=1
- 序号 (SEQ): $x+1$ (客户端上一个 SYN 序号 +1, 因为 SYN 占用 1 个序号)
- 确认号 (ACK): $y+1$ (确认收到服务器的 SYN: $y+1$, 表示「我已收到你序号 y 的 SYN, 下一个期望收到 $y+1$ 开始的数据」)
- 作用: 客户端确认服务器的 SYN-ACK 报文, 至此双方都确认了「对方的发送、接收能力正常」, 连接正式建立, 可开始传输应用数据。
- 细节: ACK 报文不携带 SYN, 因此不占用序号, 客户端后续数据报文的序号从 $x+1$ 开始递增。

Wireshark · 分组 1270 · WLAN

```
.....0. .... = LG bit: Globally unique address (factory default)
.....0. .... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 2]
> Internet Protocol Version 4, Src: 109.244.216.125, Dst: 172.28.118.66
v Transmission Control Protocol, Src Port: 8081, Dst Port: 50563, Seq: 0, Ack: 1, Len: 0
  Source Port: 8081
  Destination Port: 50563
  [Stream index: 35]
  [Stream Packet Number: 2]
  > [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 71563853
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 402226379
  1000 .... = Header Length: 32 bytes (8)
  v Flags: 0x012 (SYN, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    ....0... = Congestion Window Reduced: Not set
    ....0... = ECN-Echo: Not set
    ....0... = Urgent: Not set
    ....1... = Acknowledgment: Set
    ....0... = Push: Not set
    ....0... = Reset: Not set
    > ....1... = Syn: Set
    ....0... = Fin: Not set
  [TCP Flags: .....A..S.]
  Window: 42340
  [Calculated window size: 42340]
```

序号

确认号是SYNACK加1

ACK位置1

五、分析 ICMP 协议

```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS C:\Users\83478> ping www.baidu.com

正在 Ping www.a.shifen.com [183.240.99.224] 具有 32 字节的数据:
来自 183.240.99.224 的回复: 字节=32 时间=15ms TTL=52
来自 183.240.99.224 的回复: 字节=32 时间=21ms TTL=52
来自 183.240.99.224 的回复: 字节=32 时间=12ms TTL=52
来自 183.240.99.224 的回复: 字节=32 时间=15ms TTL=52

183.240.99.224 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 12ms, 最长 = 21ms, 平均 = 15ms
PS C:\Users\83478>
```

No.	Time	Source	Destination	Protocol	Length	Info
16218	171.270029700	172.24.183.114	183.240.99.224	ICMP	74	Echo (ping) request id=0x0001, seq=962/49667, ttl=128 (reply in 16219)
16219	171.284857100	183.240.99.224	172.24.183.114	ICMP	74	Echo (ping) reply id=0x0001, seq=962/49667, ttl=52 (request in 16218)
16242	172.279348200	172.24.183.114	183.240.99.224	ICMP	74	Echo (ping) request id=0x0001, seq=963/49923, ttl=128 (reply in 16244)
16244	172.300268000	183.240.99.224	172.24.183.114	ICMP	74	Echo (ping) reply id=0x0001, seq=963/49923, ttl=52 (request in 16242)
16268	173.291600600	172.24.183.114	183.240.99.224	ICMP	74	Echo (ping) request id=0x0001, seq=964/50179, ttl=128 (reply in 16269)
16269	173.303926800	183.240.99.224	172.24.183.114	ICMP	74	Echo (ping) reply id=0x0001, seq=964/50179, ttl=52 (request in 16268)
16304	174.306378000	172.24.183.114	183.240.99.224	ICMP	74	Echo (ping) request id=0x0001, seq=965/50435, ttl=128 (reply in 16305)
16305	174.321512300	183.240.99.224	172.24.183.114	ICMP	74	Echo (ping) reply id=0x0001, seq=965/50435, ttl=52 (request in 16304)

PowerShell 终端呈现的是 Ping 指令的整体交互结果，清晰展示了目标主机（183.240.99.224）的连通性、往返延迟与丢包情况，包括域名解析结果、32 字节的默认数据包大小、12-21ms 的往返时间（RTT）、TTL=52 的生存时间值，以及最终 0% 丢包、平均 15ms 的统计信息；而 Wireshark 抓包则还原了 Ping 指令底层的 ICMP 协议报文细节，清晰呈现出 4 组 Echo Request（Type=8，请求报文）和 Echo Reply（Type=0，应答报文）的一一对应交互，通过序列号（Seq）匹配请求与应答对，还能看到 ICMP 协议无端口、基于 IP 封装传输的特征，以及精准的报文收发时间差，对应终端显示的延迟时间。二者一个是终端层面的结果汇总，一个是网络层面的协议细节还原，共同构成了对 Ping 通信过程的完整认知 PowerShell 终端呈现的是 Ping 指令的整体交互结果，清晰展示了目标主机（183.240.99.224）的连通性、往返延迟与丢包情况，包括域名解析结果、32 字节的默认数据包大小、12-21ms 的往返时间（RTT）、TTL=52 的生存时间值，以及最终 0% 丢包、平均 15ms 的统计信息；而 Wireshark 抓包则还原了 Ping 指令底层的 ICMP 协议报文细节，清晰呈现出 4 组 Echo Request（Type=8，请求报文）和 Echo Reply（Type=0，应答报文）的一一对应交互，通过序列号（Seq）匹配请求与应答对，还能看到 ICMP 协议无端口、基于 IP 封装传输的特征，以及精准的报文收发时间差，对应终端显示的延迟时间。二者一个是终端层面的结果汇总，一个是网络层面的协议细节还原，共同构成了对 Ping 通信过程的完整认知

实验结果：

实验内容	实验操作	核心结果
Wireshark 软件安装与学习	从官方网站下载软件并安装，熟悉软件主界面、接口列表、显示过滤器与捕获过滤器的使用，了解数据包详细信息的分层展示方式	成功安装 Wireshark 软件，掌握接口选择、数据包捕获方法，能使用过滤器按协议、IP、端口等条件筛选数据包，理解数据包的五层协议分层结构
HTTP 协议抓包与分析	在浏览器输入目标网址，使用 Wireshark 捕获 HTTP 数据包，通过显示过滤器“http”筛选相关报文，查看报文详细信息	成功捕获 HTTP 请求与响应报文，明确通信的源 IP（172.28.165.166）与目的 IP（172.31.221.68），能识别 HTTP 请求方法（如 GET）及请求头信息，确认 HTTP 协议基于 TCP 端口 80 传输
TCP 协议分析	筛选 TCP 协议数据包，查看源端口、目的端口、序列号、确认号、标志位、窗口大小等关键字段，分析 TCP 流	捕获的 TCP 报文源端口为 59614（客户端随机端口），目的端口为 80（HTTP 标准端口）；头部长度 20 字节，序列号 497233153，确认号 8089，标志位为 0x018（PSH、ACK），窗口大小 256 字节，携带 714 字节 HTTP 响应数据，实现可靠传输与流量控制
TCP 三次握手分析	捕获 TCP 连接建立阶段的报文，识别 SYN、SYN-ACK、ACK 三种报文，分析各报文的标志位、序列号与确认号	成功捕获三次握手完整报文：第一次握手（SYN=1，ACK=0），客户端发送初始序列号；第二次握手（SYN=1，ACK=1），服务器同步自身序列号并确认客户端请求；第三次握手（SYN=0，ACK=1），客户端确认服务器响应，连接正式建立，明确序列号与确认号的递增逻辑
ICMP 协议分析	在 PowerShell 中执行 Ping 命令测试目标主机连通性，同时用 Wireshark 捕获 ICMP 报文，对比终端结果与抓包细节	Ping 命令成功连通目标主机（183.240.99.224），发送 4 个数据包，接收 4 个，0%丢包，往返延迟 12-21ms，平均 15ms；抓包显示 4 组 ICMP 交互报文，Echo Request（Type=8）与 Echo Reply（Type=0）一一对应，通过序列号匹配请求与应答，确认 ICMP 协议无端口、基于 IP 封装传输的特征

本次实验通过 Wireshark 软件完成数据包抓取与分析，依次对 HTTP、TCP、ICMP 协议及 TCP 三次握手过程进行探究，结合终端操作与抓包细节，得到以下实验结果，具体汇总如下：

综上，本次实验顺利完成所有既定内容，成功抓取并分析了各类协议数据包，验证了 HTTP、TCP、ICMP 协议的工作机制，掌握了数据包抓取与分析的基本方法

实验小结:

(实验中出现问题解决方法, 实验心得体会等)

通过本次数据包抓取与分析实验, 我切实掌握了 Wireshark 软件的基本使用方法, 也对 HTTP、TCP、ICMP 等网络协议有了更直观的认识, 不再是书本上抽象的理论, 而是能通过抓包看到真实的网络数据交互过程, 收获很多也很有成就感。

实验一开始, 我按照指导步骤从 Wireshark 官方网站下载并安装软件, 过程很顺利, 一路点击“下一步”就能完成。打开软件后, 面对众多网络接口和复杂的界面, 我一开始不知道该如何下手抓包。后来慢慢摸索, 学会了选择对应的网络接口, 点击开始捕获, 看着屏幕上不断刷新的数据包, 逐渐明白了每个字段的大致含义, 也学会了使用显示过滤器筛选需要的协议, 比如输入 http 就能只看到 HTTP 协议的数据包, 输入 icmp 就能找到和 Ping 相关的报文。

在分析 TCP 协议和三次握手时, 一开始对序列号、确认号和标志位的理解不够透彻, 看着抓包数据里的数字和符号, 一时不知道如何解读。后来结合实验指导, 反复对照 Wireshark 里的报文细节, 一点点理清了 SYN、SYN-ACK、ACK 三种报文的作用, 也明白了三次握手是如何确保网络连接可靠建立的, 原来那些看似复杂的数字, 都是为了保证数据能有序、完整地传输。

分析 ICMP 协议时, 我通过在 PowerShell 里输入 Ping 命令, 再用 Wireshark 抓包对比, 清楚地看到了 Ping 指令背后的 ICMP 报文交互过程。终端上显示的往返延迟、丢包率, 在抓包里都能找到对应的报文细节, 让我明白了 Ping 指令其实是依靠 ICMP 协议实现的, 也理解了两者的关系。

整个实验过程中, 我也遇到了一些小问题, 比如一开始不会筛选特定协议的数据包, 导致抓包后找不到需要分析的内容, 后来通过仔细看实验步骤和软件界面, 慢慢掌握了过滤器的使用方法; 还有在解读 TCP 报文的标志位时, 容易混淆不同标志的含义, 通过反复对照和思考, 终于弄明白了每个标志的作用。

、

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月

日

备注：